Fundamental Programming Structures in JAVA

INTERFACE

Objectives

- To understand the concept of Java interfaces.
- To outline the benefits of using interfaces.
- To define a Java interface.
- To learn how to use Java interfaces.

Interface

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body.
- In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Interface

- An interface is a programming construct in Java that defines a **contract** (a set of behaviours) that classes must follow.
- An interface defines a set of abstract methods (methods without bodies) that must be provided by any class that implements the interface.
- In result, Interfaces allow you to create a contract that ensures all implementing classes use the same method signatures.

Benefits of Interface

- The intent of the interface is to specify a common behavior for objects.
- Interfaces provide a powerful tool for accomplishing Abstraction, multiple Inheritance and Polymorphism.
- Interfaces are also used to provide code reusability and support good software engineering practises.

How to use Java interfaces?

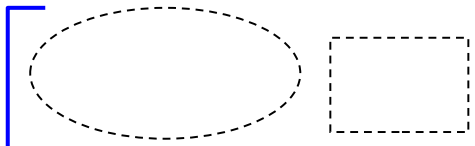
- An interface is treated like a special class in Java.
- Each interface is compiled into a separate bytecode file, just like a regular class.
- Like an abstract class, you cannot create an instance from an interface using the new operator, but in most cases you can use an interface more or less the same way you use an abstract class.
- For example, you can use an interface as a data type for a variable, as the result of casting, and so on.

Define a Java interfaces

- An **interface** are class-like structures that contain only constant data fields and abstract methods.
- **Interface contains:**
  - only abstract methods (without adding the “abstract” keyword).
  - **Static final** fields
  - no need to constructors too.
  - A class that derives from an interface must **implement** all members of the interface.
  - To denote that the subclass inherits an interface, (implements) keyword should be used.

Interface Template

A class Template



Class Name

List of Attributes …..

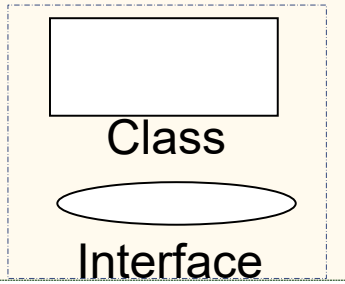
$Method_1$ name (): return data type or void
$Method_2$ name (): return data type or void
$Method_3$ name (): return data type or void
.
.
$Method_3$ name (): return data type or void

Interface name

List of Attributes……..

$Method_1$ name (): return data type or void ;
$Method_2$ name (): return data type or void ;
$Method_n$ name (): return data type or void ;

symbols used in class design

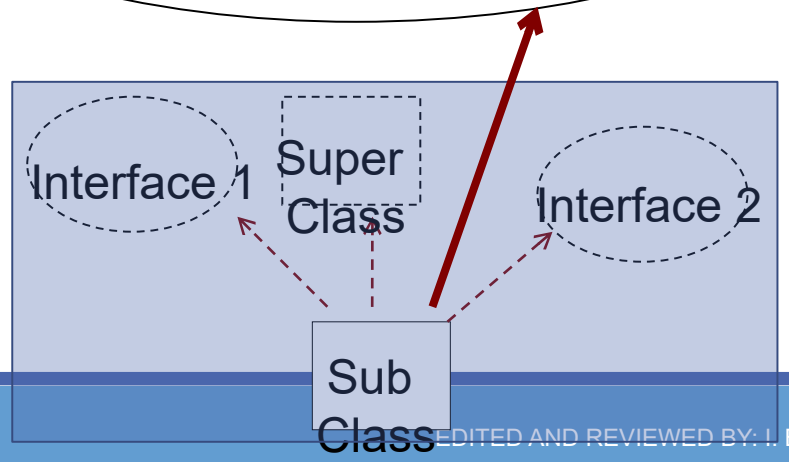


Interface Template

Interface name

List of Attributes……..

Method$_1$ name (): return data type or void ;
Method$_2$ name (): return data type or void ;
.
.
Method$_n$ name (): return data type or void ;



Regardless of whether a programmer specifically writes a static final, any data field added to an interface will be regarded as a static final; therefore, the value of these data fields should be provided directly in the declaration statement.

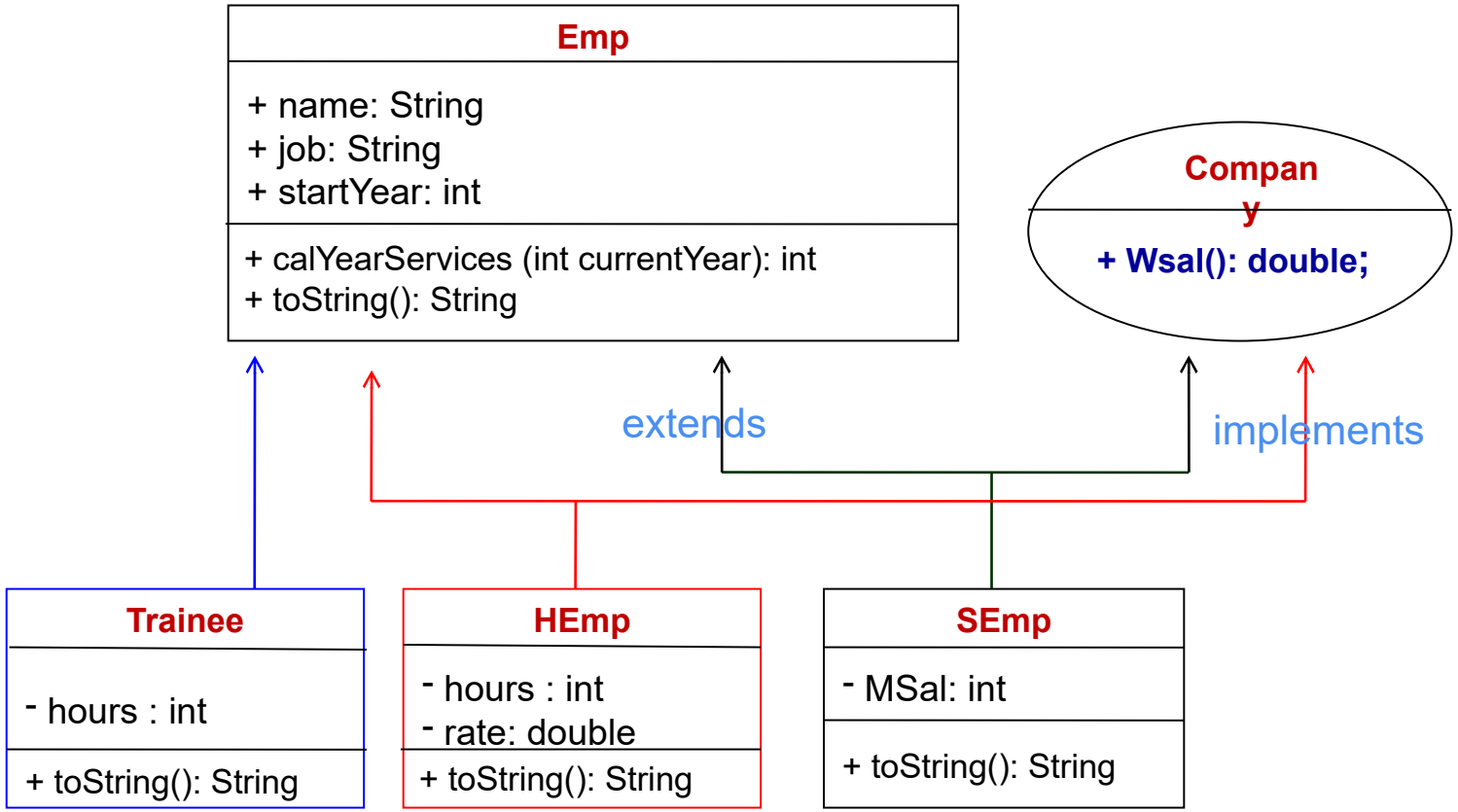
The special thing about the abstract class is that it has only abstract methods (either with or without adding the “abstract” keyword at the implementation phase) by adding a semicolon at the end of each method declaration during the design stage.

Remember that a subclass can extends only one concrete class or an abstract class, but it can implements any number of interfaces..

Interfaces Example

- **Example**: Going back to the company that has two types of employees, the first type gets a monthly salary (**SEmp**), and the other gets an hourly-based salary (**HEmp**). This company also employs (**Trainees**) who work for experience only and receive no pay from the employer.

Interfaces Example (Design Phase)



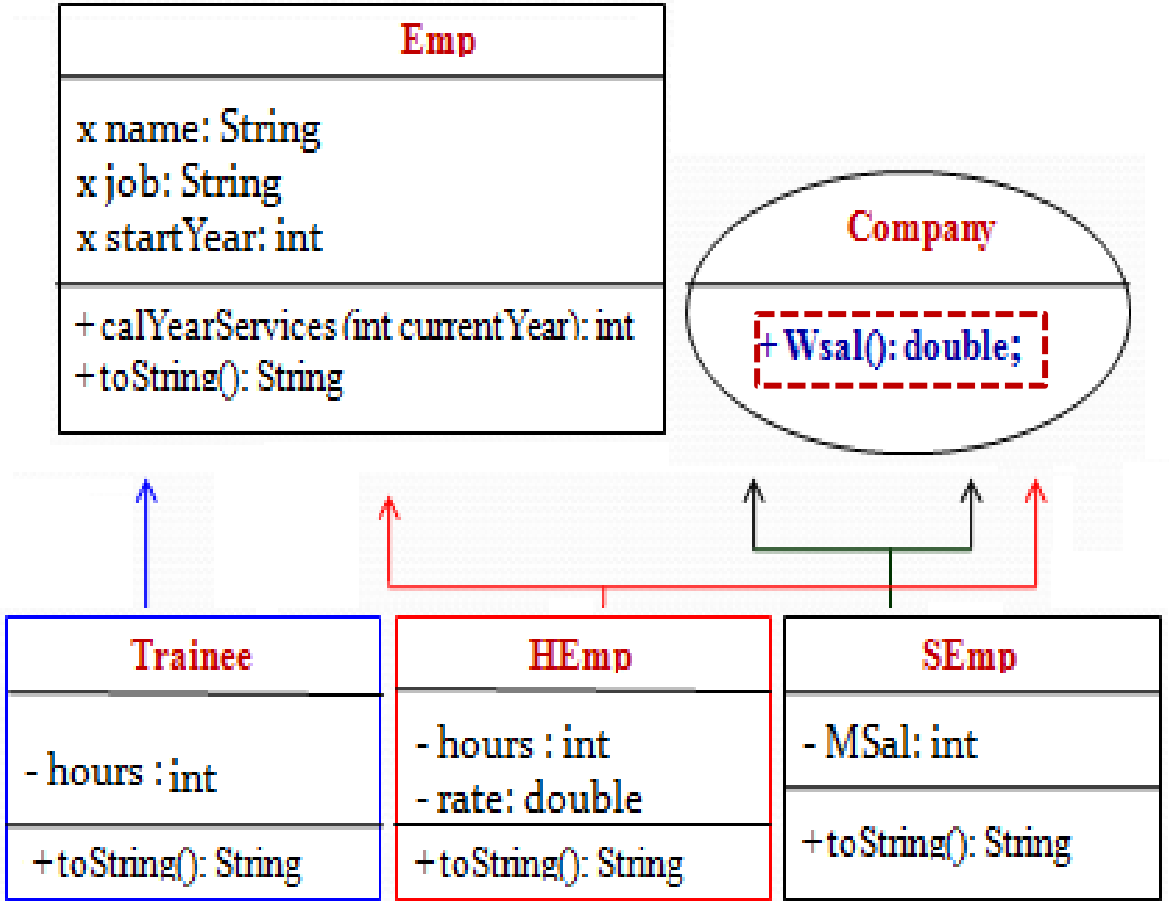
Interfaces Example

When interface is preferable solution?

- When this method is applicable to some subclasses, and no need to add this method to the super class.
- When the method is shared between many classes but they can't be subclass from a specific class (no sense).

For example, there is a method that calculates the employee's monthly salary called wsal(). If you put it in the superclass (**Emp**), then the company must pay salaries to all employees, including **SEmp**, **HEmp**, and **Trainees**. However, **Trainees** do not receive a salary; they only work for experience. Because of this, it is better to shift wsal() from the **superclass** to the interface and let **SEmp** and **HEmp** implement that interface, and both **SEmp** and **HEmp** should redefine **(override)** it in a way that suits them.

Interfaces Example (Design Phase)



- Note that the **Company interface** contains an abstract method **(Wsal()).**
- **(Wsal()) is a**n abstract method that has only signature and without body.
- subclasses (SEmp and HEmp) must override the abstract methods (**Wsal()**). Otherwise, compile-time errors will appear in all the subclasses that implement from the interface.
- **Logically**, it is not allowed for the Trainees subclass to override the Wsal() method.

Interfaces Syntax

To declare an Interfce, we use the access modifier first, then the “interface" keyword, and the interface name:

```
access_modifier interface interface_Name{ }
```

To declare a data field, we use the access modifier first, then either with or without adding the final static, the data type, and the variable name, and finally give the value of the data field directly in the declaration statement.

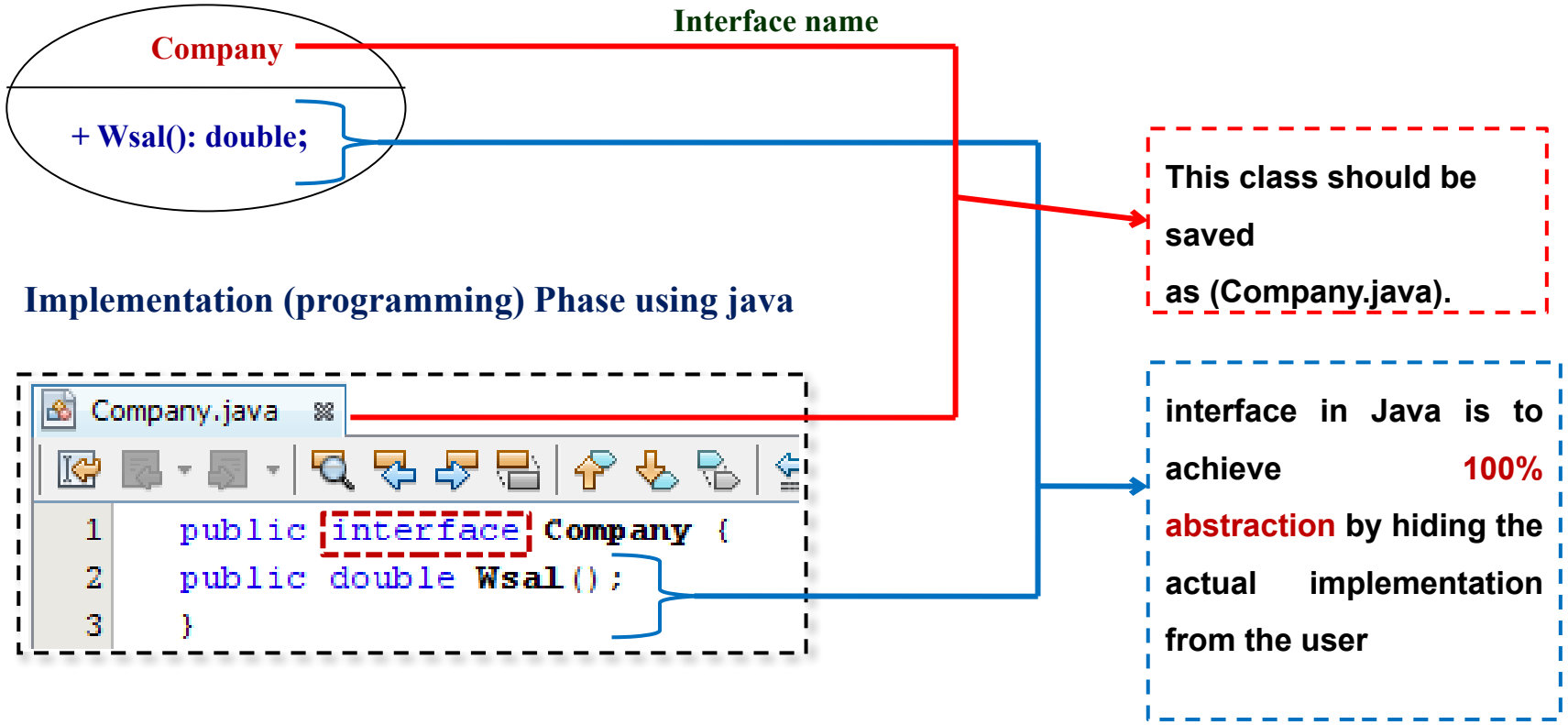
```
access_modifier final static data type variableName = value ;
```

Interfaces Syntax

- To declare an abstract method, we use the access modifier first, then either with or without adding the “abstract” keyword, the return type, the method name, and finally a semicolon at the end of the method definition.

```
access_modifier abstract return_type methodName (parameter_list);
```

Interfaces Example implementation phase



Interfaces Example implementation phase

Company.java | Emp.java

```java
class Emp  {
        protected String name;  protected String job;  protected int startYear;
        public Emp(){ name="unknown"; job="unknown"; startYear=0;        }
        public Emp(String name){ this(); this.name = name;}
        public Emp(String name, String job) { this(name);this.job=job;}
        public Emp(String name, String job, int sy){
             this(name,job); startYear=sy;}
        public int calYearServices(int currentYear)
        {return currentYear-startYear;}

        public String toString(){
                return "Name:"+name+" job:"+job+" Starting-at:"+  startYear;}

}
```

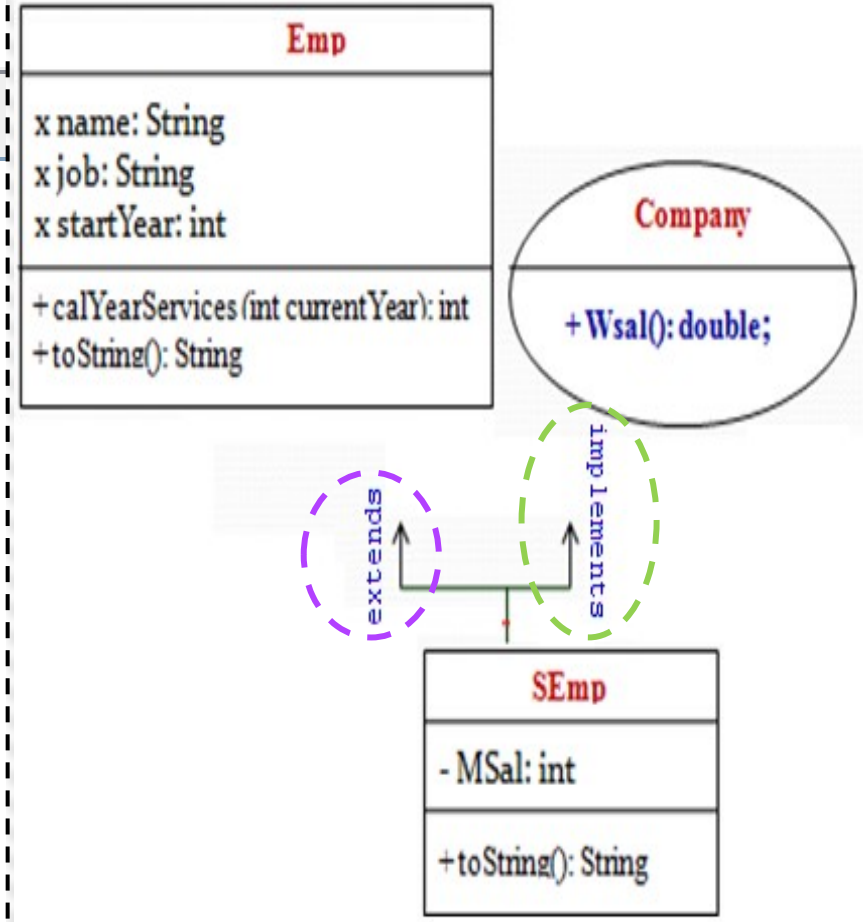
Interfaces Example implementation phase

Company.java | Emp.java | SEmp.java

```java
class SEmp extends Emp implements Company{

        private double MSal;
        public SEmp(String name, String job, int sy, double salary)
                {super(name,job,sy); this.MSal=salary;  }
        public double Wsal() {
            return MSal*12/52; }

        public String toString(){
                return super.toString()+"\n" + " His salary = "+MSal;
}
}
```



Interfaces Example implementation phase

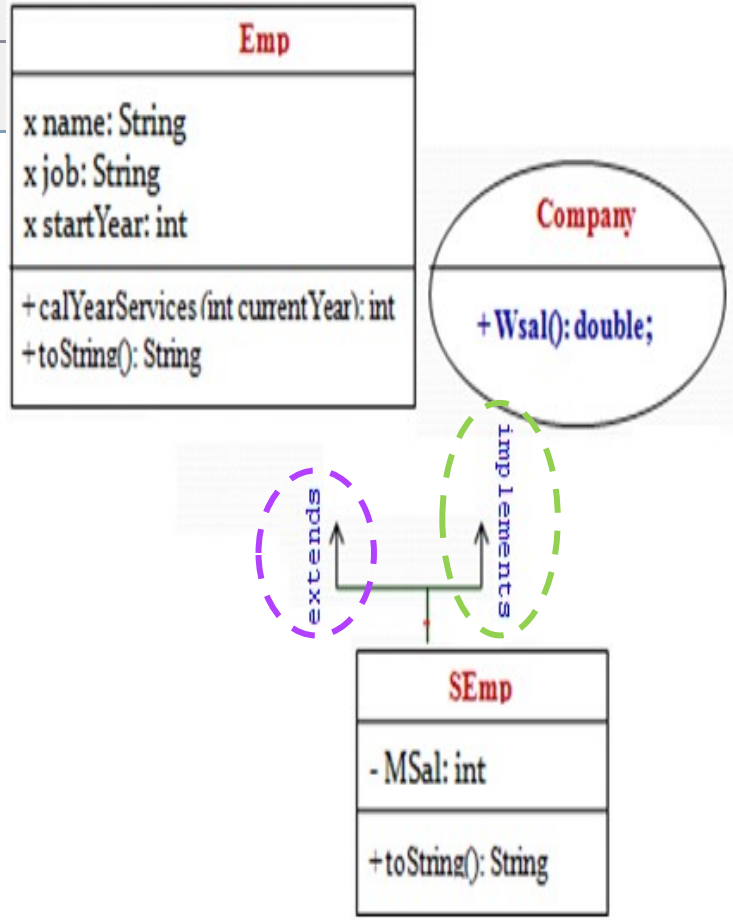
```
1   class SEmp  extends  Emp  implements  Company {
2
3           private double MSal;
4           public SEmp(String name, String job, int sy, double salary)
5                   {super(name,job,sy); this.MSal=salary;  }
6
7           public double Wsal() {
8               return MSal*12/52; }
9
10          public String toString(){
11                  return super.toString()+"\n" + " His salary = "+MSal;
12  }
13  }
```

SEmp overrides the abstract methods (Wsal()) to add the missing part (the body of the method), which is its own method for calculating the employee's monthly salary.

Interfaces Example implementation phase

Company.java | Emp.java | SEmp.java | HEmp.java

```
1   class HEmp extends Emp implements Company{
2
3       private int hours;          private double rate;
4           public HEmp(String name, String job, int sy, int hours, double rate)
5   {super(name,job,sy); this.hours=hours; this.rate=rate;  }
6
            public double Wsal() {
8             return hours*rate; }
9
            public String toString(){
11          return super.toString()+"\nNo of Hours:"+ hours+" rate="+ rate;
12  }
13  }
```



Interfaces Example implementation phase

```
1   class HEmp extends  Emp  implements  Company{
2
3       private int hours;          private double rate;
4           public HEmp(String name, String job, int sy, int hours, double rate)
5   {super(name,job,sy); this.hours=hours; this.rate=rate;  }
6
7           public double Wsal() {
8             return hours*rate; }
9
10          public String toString(){
11          return super.toString()+"\nNo of Hours:"+ hours+" rate="+ rate;
12  }
13  }
```

–HEmp overrides the abstract methods (Wsal()) to add the missing part (the body of the method), which is its own method for calculating the employee's hourly salary.

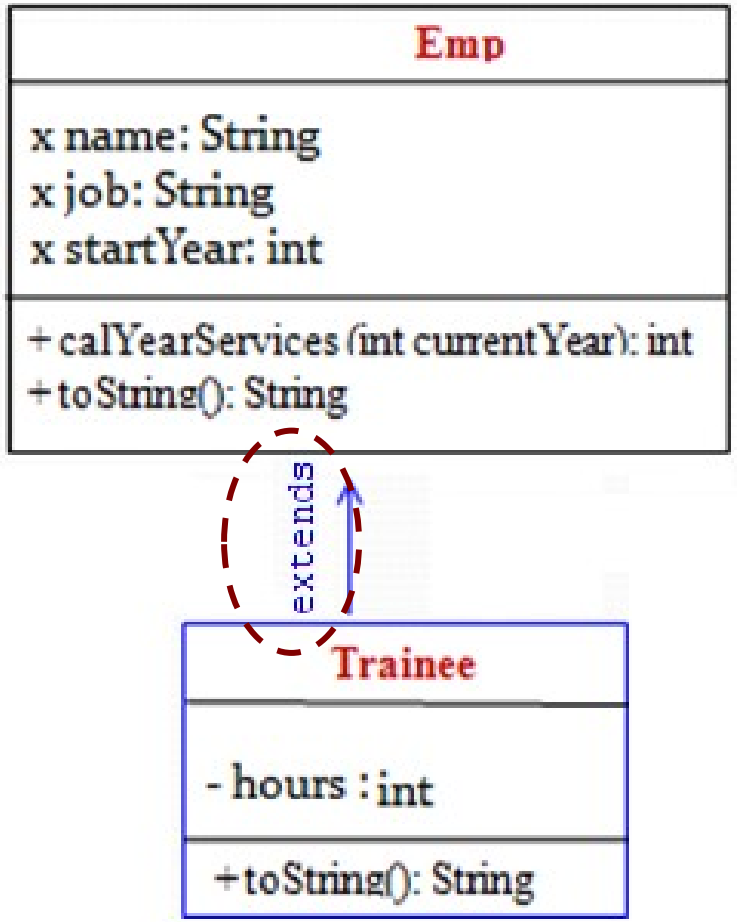
Interfaces Example implementation phase

Company.java | Emp.java | SEmp.java | HEmp.java | Trainee.java

```java
public class Trainee extends Emp {

    private int hours;

    public Trainee(String name, String job, int sy, int hours) {
        super(name, job, sy);
        this.hours = hours;
    }
public String toString(){
        return super.toString()+"\nNo of Hours:"+ hours;
}
}
```



Interfaces Example implementation phase

```java
public class Trainee extends Emp {

    private int hours;

    public Trainee(String name, String job, int sy, int hours) {
        super(name, job, sy);
        this.hours = hours;
    }
public String toString(){
        return super.toString()+"\nNo of Hours:"+ hours;
}
}
```

–**Logically**, it is not allowed for the Trainee subclass to override the Wsal() method. **But what if wsal() is defined in the trainee subclass?** In this case, no error will appear at compile-time, and the wsal() method that will be defined in the Ttrainee subclass will be considered **a new method**, not an overridden method.

Interfaces Example implementation phase

Company.java | Emp.java | SEmp.java | HEmp.java | Trainee.java | test.java

```
1  public class test {
2      public static void main(String[] args) {
3          Company z;
4          z = new Company();
```

Note that the Company interface cannot be instantiated (i.e., created as an object) directly as shown in line 4, but we can define a pointer as shown in line 3; look at the following figure to read the error.

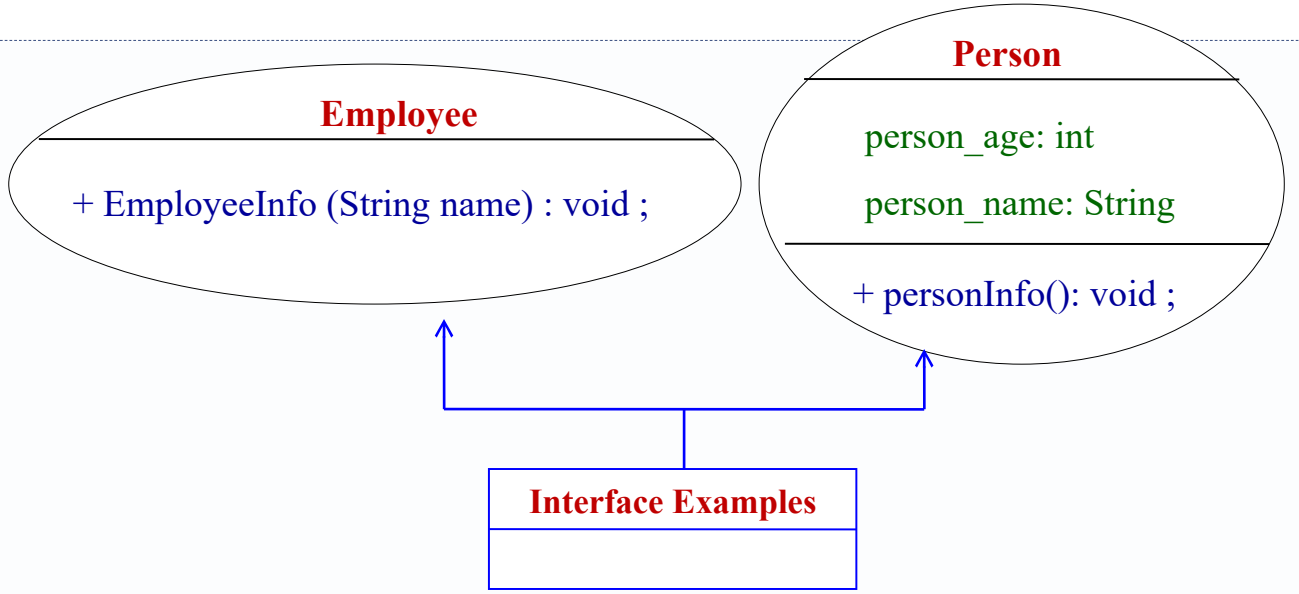
```
Company z;
z = new Company();
```

Company is abstract; cannot be instantiated

How to implement multiple interfaces in java?

Simple Example: In the following, we create two interfaces (the first one is named "**Person**" and the second one is named "**Employee**") and a class named "**InterfaceExamples**".

The "**InterfaceExamples**" class implements the interface "Person" and the interface "Employee":



How to implement multiple interfaces in java?

Employee

+ EmployeeInfo (String name) : void ;

Person

person_age: int

person_name: String

+ personInfo(): void ;

```
1   interface Employee {
2
3       public void EmployeeInfo(String name);
4
5   }
```

Method Declaration

```
2
    interface Person {
4
5       int PERSON_AGE = 32;
6
7       String PERSON_NAME = "JOHN";
8
        public void displayInfo();
10  }
```

Constant Fields

Method Declaration

The employee interface declares a method called "**EmployeeInfo()**", which takes a parameter.

The Person interface declares two constants Fields and a method called "**PersonInfo**() “.

How to implement multiple interfaces in java?

The below-given screenshot provides the complete code along with the output:

```
3   public class InterfaceExamples implements Person, Employee {
4
5       public void PersonInfo() {
6           System.out.println("Person Name : " + PERSON_NAME);
7           System.out.println("Person Age : " + PERSON_AGE);
8       }
9
10      public void EmployeeInfo(String name) {
11          System.out.println("Employee Name : " + name);
12      }
13
14      public static void main(String[] args) {
15          InterfaceExamples exp = new InterfaceExamples();
16
17          exp.PersonInfo();
18          exp.EmployeeInfo("Michael");
19      }
20  }
```

Class implements Multiple Interfaces

Method Definition of Person Interface

Method Definition of Employee Interface

Object creation

Method Calling with object of class

Omitting Modifiers in Interfaces

- All data fields are ***public final static*** and all methods are ***public abstract*** in an interface. For this reason, these modifiers can be omitted, as shown below:

```
public interface T1 {
  public static final int K = 1;

  public abstract void p();
}
```

Equivalent

```
public interface T1 {
  int K = 1;

  void p();
}
```

- A constant defined in an interface can be accessed using syntax ***InterfaceName.CONSTANT_NAME*** (e.g., ***T1.K***).

Abstract Class Vs Interface

The following table shows a comparison between interface and abstract class.

Abstract class	**Interface**
Abstract class …. { }	Interface ….. { }
May contain abstract method	All methods are abstract without saying that
extends	implements
All type fields	Static final fields
Can not be instantiated	
All sub classes must override all abstract method	

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual